



2025 TRAVELERS UMC

UConnFIDENCE INTERVAL

November 24, 2025

Team – UConnfidence Interval



**Carlos
Correa**

Education:

Miami University
(Ohio)

Experience:

Data Engineering
Intern at
Constellation
Brands



**Callum
Reilly**

Education:

University of
Connecticut



Yeongeun Ra

Education :

Chung-ang
University

Experience :

AI trainee @ LG AI
Research



**Anupama
Rajendran**

Education:

MG University

Experience:

Back End Developer
at TCS
Full Stack Software
Developer Intern
at RBC



**Justin
Isaac**

Education:

California State University
Monterey Bay

Experience:

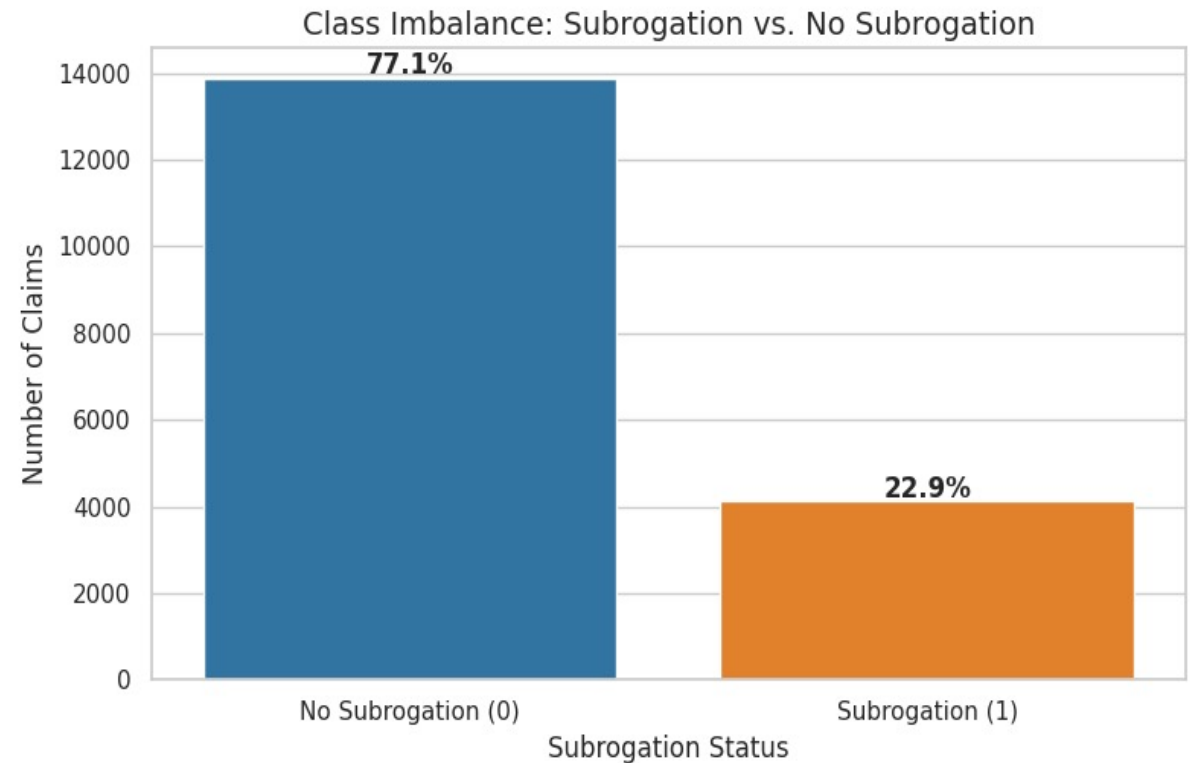
ML Intern at Codeveda
Technologies

Agenda

1. Data & Preprocessing
2. Modeling Approach
3. Variable Selection
4. Model Results & Evaluation
5. Future Improvement

1. Data & Preprocessing

- We first took a look at the distribution of subrogation opportunities in the training set:
 - 77.1376% 0's
 - 22.8624% 1's
- Handling Class Imbalance:
 - Implemented `scale_pos_weight`:
 - `scale_pos_weight = (y_clean == 0).sum() / (y_clean == 1).sum()`
 - Forces our model to penalize missing positive cases more heavily



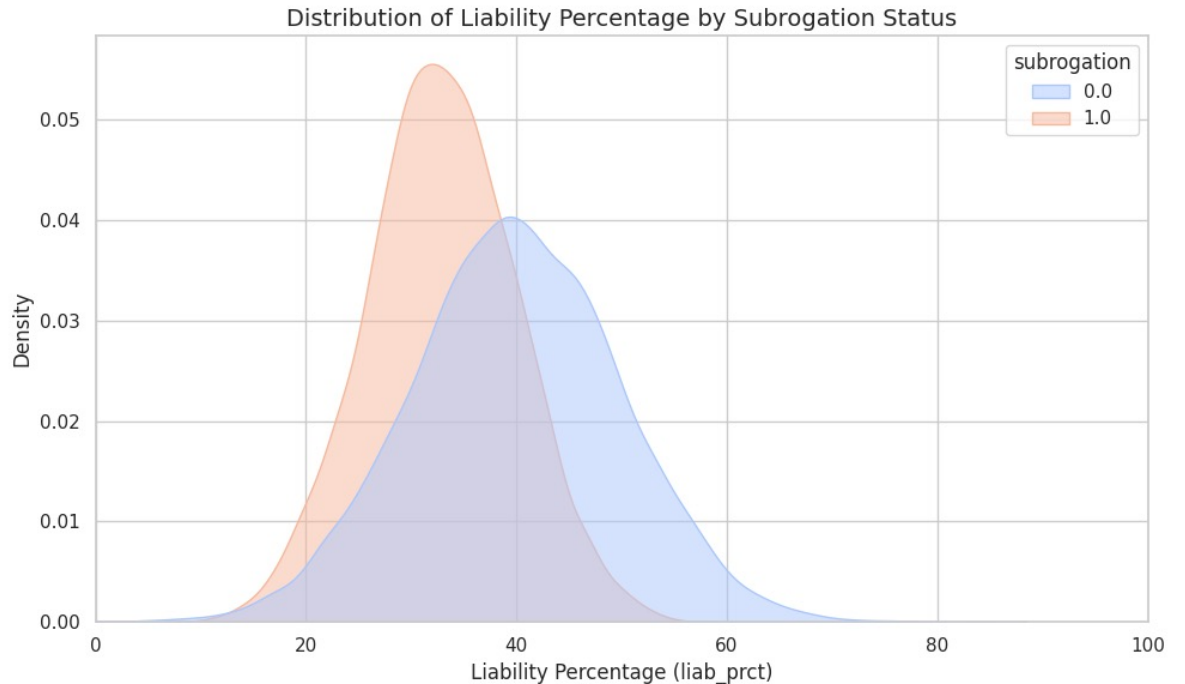
1. Data & Preprocessing

- Key Predictor: **liab_prct**

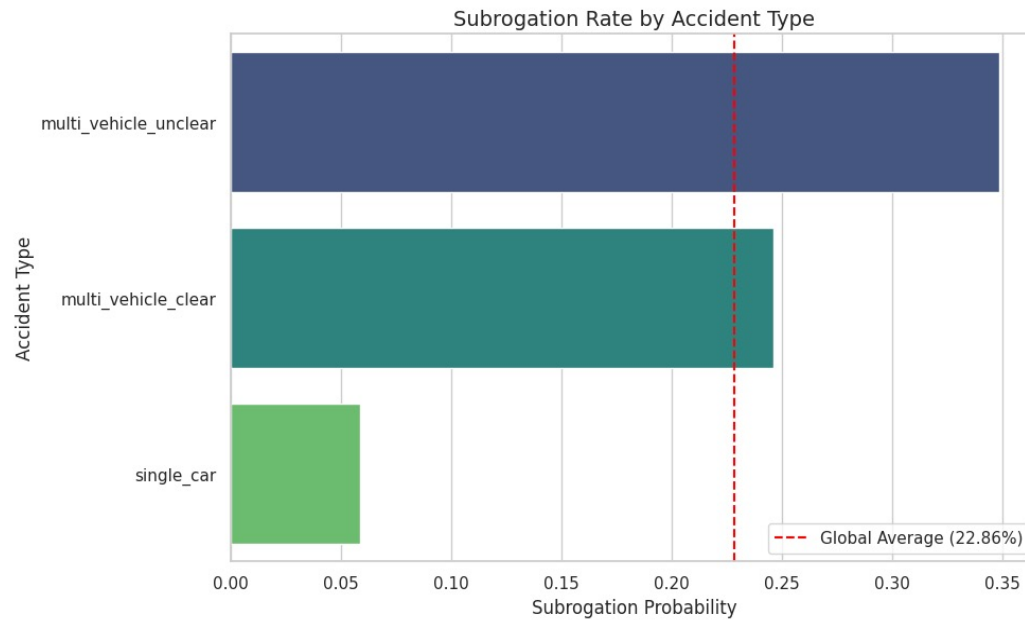
- Subrogation cases exist nearly exclusively in the 0%-55% liability range
- So, when **liab_prct** exceeds ~50%, we know that the likelihood of subrogation is effectively zero.

This insight influenced us to create the new feature **'is_not_at_fault'** for claims with a liability percentage under 50%:

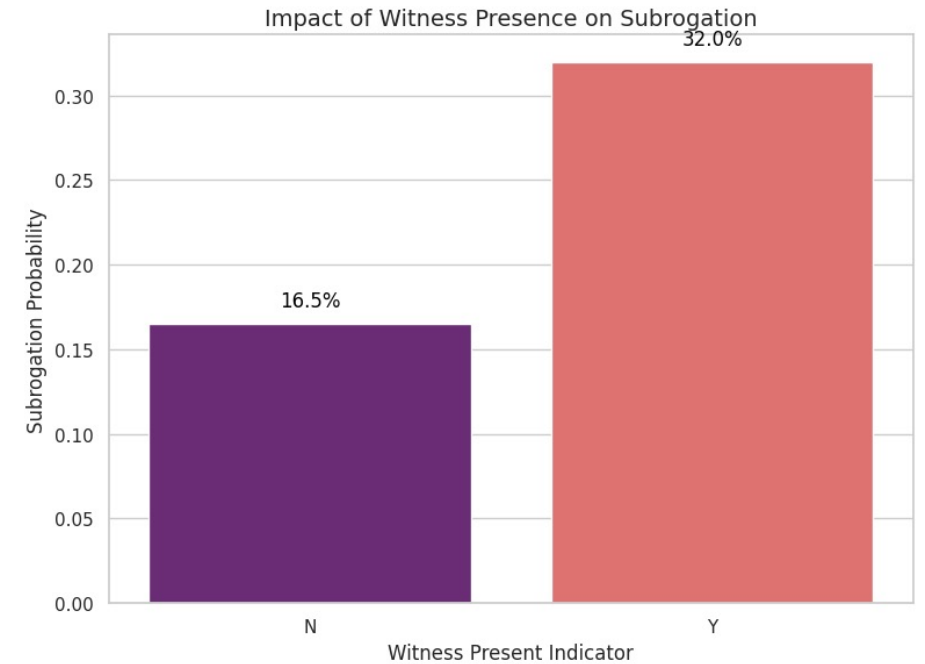
```
combined_df['is_not_at_fault'] =  
(combined_df['liab_prct'] < 50).astype(int)
```



1. Data & Preprocessing:



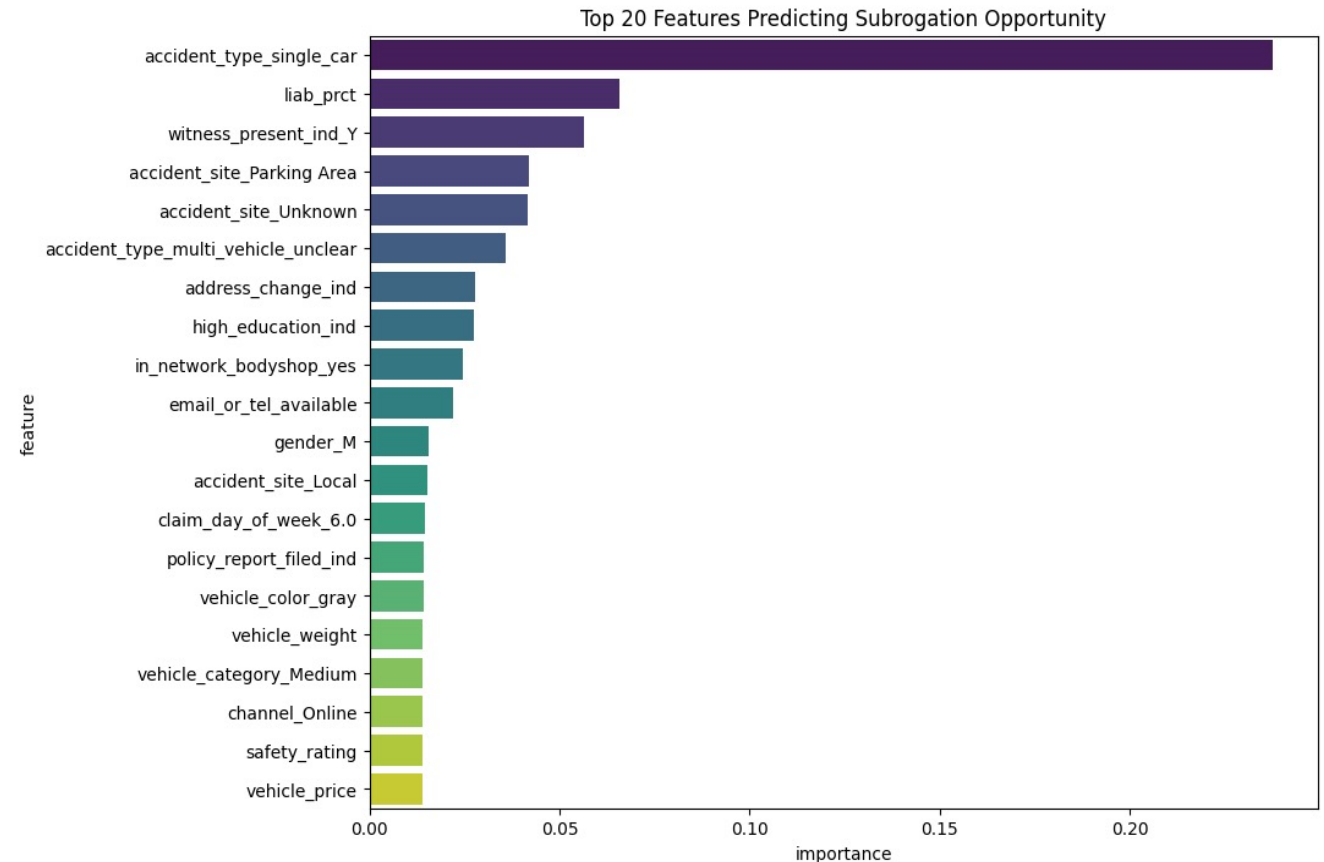
Accident Type



Witness Presence

1. Data Preprocessing

- Initial XGBoost Model Feature Importance:
 - Highlights importance of single car accidents, liability percentage, and the presence of a witness in subrogation claims
 - Served as the inspiration for the creation of many new engineered features



1. Data Preprocessing

- Feature Engineering:

- Throughout our process of fitting new models, we created the following useful features:
- `combined_df['has_strong_evidence'] = ((witness_num == 1) | (combined_df['policy_report_filed_ind'] == 1)).astype(int)`
- `combined_df['not_fault_and_evidence'] = combined_df['is_not_at_fault'] * combined_df['has_strong_evidence']`
- `combined_df['not_fault_multi_vehicle'] = ((combined_df['not_fault_and_evidence'] == 1) & (combined_df['accident_type'] != 'single_car')).astype(int)`
 - + Many more!

2. Modeling & Approach

Modeling Approach

- Neural Networks
 - Deep Neural Networks
- Tree Based
 - Random Forest
 - XgBoost
 - LightGBM
 - CatBoost

Neural Networks

We used TensorFlow to create out neural networks

Test different:

- Number of layers
- Nodes in a layer
- Activation Functions

Trees

Why Trees?

- Tabular & structured data fits tree models better
- Handles nonlinearity well
- Provides interpretable importance metrics
- Better performance in cross-validation

Trees – Random Forest

Model Performance: **Random Forest**

- Performed worse in cross-validation compared to boosting models.
- Resulted in low F1-score.

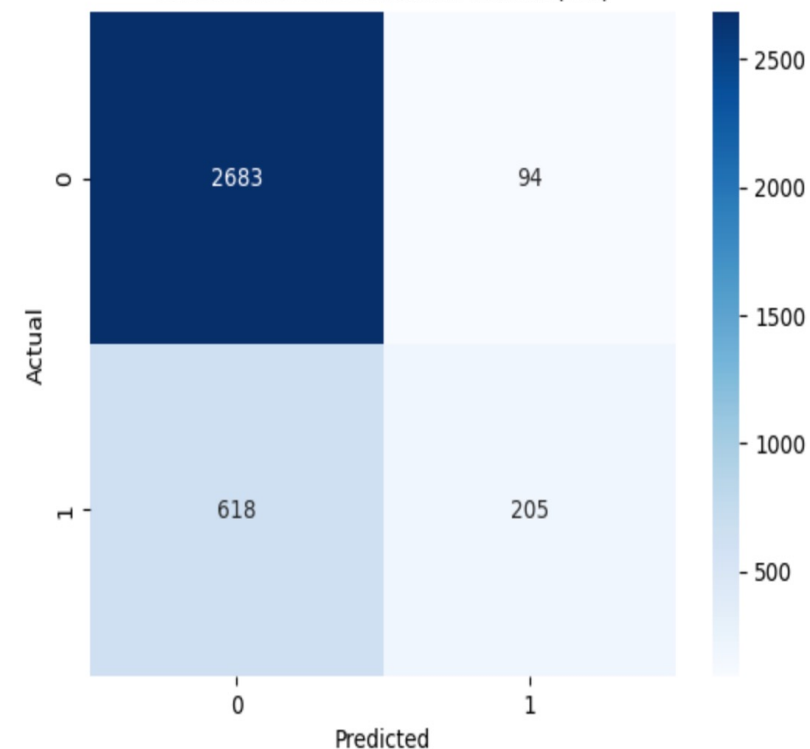
Characteristics:

- Struggles with imbalanced data.
- Inconsistent performance across folds.

RandomForest metrics:

Accuracy: 0.8022, Precision: 0.6856, Recall: 0.2491, F1: 0.3654, ROC-AUC: 0.8125

RandomForest Confusion Matrix (val)



Trees – XGB, LGB, CB

Test XGB, LGB, CB + XGB, LGB, CB + model ensemble

→ **Final model: XGBoost**

Testing Assumption

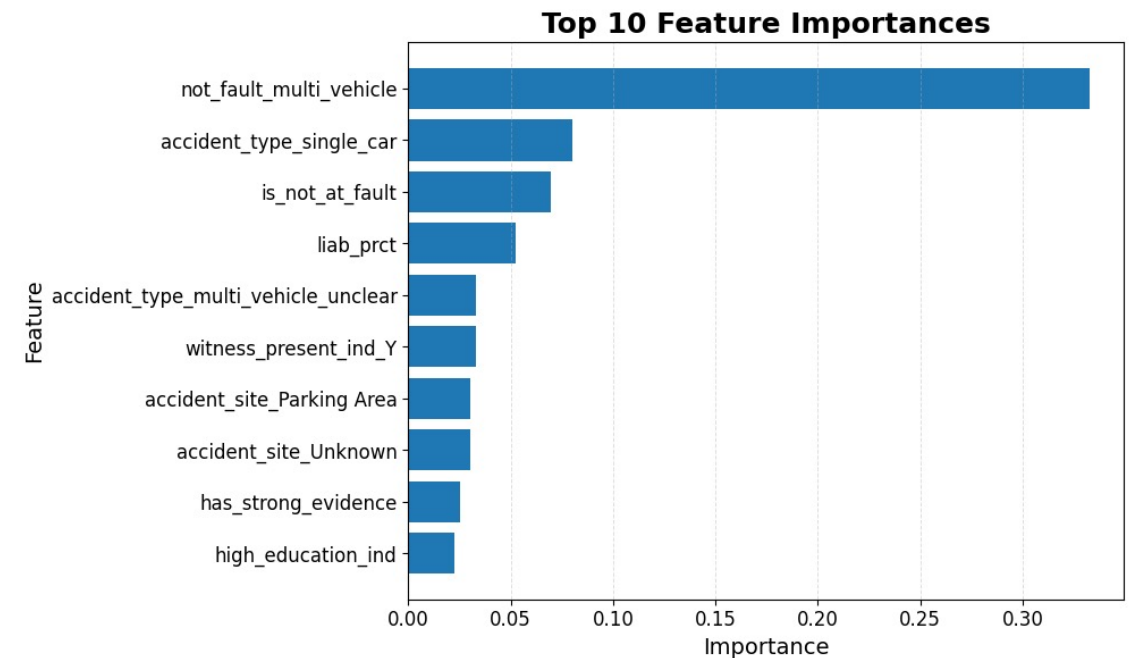
- No strict statistical assumptions required
- Validation model stability with K-fold cross-validation
- Hyperparameter tuning(Optuna) for complexity control

3. Variable Selection

- Used SHAP values to identify low-importance features
- Dropped unnecessary columns during feature engineering:
 - claim_number
 - claim_date,
 - year_of_born,
 - vehicle_made_year
 - zip_code.

Final Feature importance

- Top predictors:
engineered features : fault- related variables
& accident type and context



4. Model Results & Evaluation

Fit Statistics – Success Metrics:

Fit Statistics	Detail
F1 -score	• Subrogation label is minority class • Balances Precision and recall for model
Optimal threshold	• Performed threshold search • Using the Precision-Recall curve – get threshold that maximizes F1 score for each fold
Performance	• Robust performance across five different validation - each F1 scores across the five folds were tightly clusters. • Final score - Mean F1 across folds

4. Model Results & Evaluation

Controlling Overfitting

Levels	Detail
Stratified K fold validation	Ensures that the minority class is represented equally in every test fold
Hyperparameter Tuning	Optuna – searched best combination of hyperparameters

Model Results

- Mean cross-validation F1 score: 0.5967
- Optimal prediction threshold: 0.5426
- Private score: 0.5824
- Public score: 0.6091

5. Future Improvement

Model Improvement:

- Trying different models like a TabNet
- More in-depth feature engineering

Not Collected variables that would help analysis:

- If the car was totaled
- Crash Severity
- Police Report Summary

Thank You
Any Questions?